PROJECT REPORT

FOOD HEALTHINESS CLASSIFIER

Submitted by : Anishka Narang

Registration No. : 25BAI10513

Course : Fundamentals of AI and ML

PROBLEM STATEMENT

Problem

- In the modern food industry, nutritional labels are often complex and misleading. Consumers struggle to quickly determine if a product is "Healthy" or "Unhealthy" based on raw numbers like grams of sugar or milligrams of sodium.

Solution Goal (Objective)

- To build a binary classifier that automates this decision-making process. The model learns to weigh different nutrients (Calories, Fat, Sugar, Sodium, and Protein) to provide a definitive health classification.

APPROACH AND METHODOLOGY

This project utilizes a **Perceptron Linear Classifier**, one of the foundational algorithms in Machine Learning.

- **Algorithm Selection:**

  I chose the Perceptron because it demonstrates the fundamental "Neuron" logic: taking multiple inputs, applying weights, and producing a binary output.

- **Pure Python Implementation:**

  To showcase a deep understanding of the mathematics involved, the model was built from scratch without using high-level libraries like scikit-learn or numpy.

- **Feature Engineering:**

  Five features were selected based on standard 100g nutritional labels:

  - Calories (Energy density)
  - Total Fat (Lipid content)
  - Sugar (Glycemic impact)
  - Sodium (Salt/Preservative levels)
  - Protein (Muscle-building/Satiety factor)

KEY DECISIONS AND LOGIC

The Weighting System:

The model starts with zero knowledge (weights = 0.0). During training, it uses the **Perceptron Learning Rule**:

$$w_{new} = w_{old} + learning_rate * (y_{true} - y_{pred}) * x$$

Bias Term:

A bias (b) was included to shift the decision boundary away from the origin, allowing the model to be more flexible in its classification.

Data Storage:

Instead of complex databases, I implemented a simple text-based persistence system (weights.txt) to save the "learned" intelligence of the model.

IMPLEMENTATION DETAILS

- **Language:** Python 3.x

- **Architecture:**
  - **train mode:** Iterates through training data, calculates errors, and updates weights.
  - **predict mode:** Loads saved weights and applies the dot-product formula to new user input.

- **Command Line Interface:** The project is fully executable via terminal, meeting the course requirement for non-GUI accessibility.

CHALLENGES AND LEARNINGS

Data Normalization:

A major challenge was the difference in scales (e.g., Sodium is measured in hundreds, while Fat is in single digits). I addressed this by keeping the learning rate low (0.01) to prevent weight "explosions."

Optimization:

Without math or numpy, implementing the dot-product required manual loops. This highlighted the computational efficiency that specialized libraries usually provide.

Key Learning:

I learned that AI is essentially a mathematical optimization problem. Seeing the weights change from 0 to negative values for "Sugar" and positive values for "Protein" visually proved how the model learns human-like preferences.

CONCLUSION

- The Food Healthiness Classifier successfully demonstrates that complex health decisions can be modeled using simple linear algebra.
- By building this without external modules, the project proves that AI fundamentals are accessible through pure logic and code.

REFERENCES

- Python official documentation: https://docs.python.org/3/
- pandas documentation: https://pandas.pydata.org/docs/
- matplotlib documentation: https://matplotlib.org/stable/contents.html
- Vityarthi Python Essentials Course materials, VIT
- Various online Python programming tutorials and examples